

Win-Agent: A Multi-Agent Framework for Windows GUI Automation with Hybrid Lightweight Vision-Enhanced Perception

Yao Liu
School of Computer
Engineering and Science
Shanghai University
Shanghai, China
liuyao_1585@shu.edu.cn

Wenhao Zhu*
School of Computer
Engineering and Science
Shanghai University
Shanghai, China
whzhu@shu.edu.cn

Rui Fan
Yanfeng International Automotive
Technology Co., Ltd
Shanghai, China
rui.fan@yanfeng.com

Abstract—Multi-modal Large Language Model (MLLM)-driven GUI agents hold significant potential for automating tasks on Windows user interfaces. However, due to inherent technical limitations of MLLMs, these agents still face challenges in accurately perceiving GUI elements, particularly in complex scenarios like Windows. The Windows environment also often involves more intricate cross-application tasks, which impose higher demands on the decision-making capabilities of GUI agents. To address these challenges, this paper proposes Win-Agent, a multi-agent framework. At the perception level, we design a Multi-Dimensional Perception Module (MDPM) that enhances the agent's visual perception of the interface through collaboration between small and large models, eliminating reliance on accessibility trees. At the decision-making level, we introduce a planning-execution-reflection multi-agent framework to effectively handle complex user requests and dependencies among subtasks in Windows. Furthermore, we construct a new benchmark, Windows-Eval, comprising 150 complex user requests from real-world scenarios. Experimental results show that Win-Agent achieves an absolute improvement of 6% in task success rate over prior representative methods.

Index Terms—GUI Automation, MLLM Agent, Multi-Agent

I. INTRODUCTION

A Large Language Model (LLM) Agent is an AI system with an LLM as its core reasoning engine. It combines abilities like environment sensing, state memorization, and task execution [1]. The flourishing development of Multi-modal Large Language Models (MLLMs) is driving the study of next-generation agents. In this field of research, GUI agents have gained significant interest. The primary goal of such agents is to automate the control of devices such as PCs, so that complicated procedures can be executed through simple natural language instructions. This research direction holds substantial research value and broad application prospects. Furthermore, if an agent can operate screens and use these devices seamlessly according to users' natural language instructions, it would mark a critical step forward toward general intelligence [2].

The Windows operating system is a representative OS for personal computers [3]. Despite a clear need for developing

GUI agents for Windows, this area remains underexplored, with several key issues to address. First, existing agents primarily focus on smartphones [4] or web applications [5]. In contrast, the Windows environment often involves more complex, multi-step tasks that span across different applications. A single task may comprise multiple subtasks within various applications. The lengthy action sequences and dependencies between subtasks can make it difficult for a single, unoptimized agent to effectively plan and reason through these complex tasks. As illustrated in Fig. 1, the success rate of a single agent approach for complex tasks is markedly lower than that of multi-agent frameworks. Second, one essential capability for an effective GUI agent is the visual grounding of GUI elements [6]–[9]. However, a mismatch exists between the dense nature of screen coordinates and the coarse-grained image patch features extracted by models like Vision Transformers [6]. This leads to limitations in the ability of MLLM-driven GUI agents to accurately locate GUI elements. To address these issues, existing research has proposed solutions such as UFO [10], which introduces a dual-agent framework where one agent handles global planning and application switching, and the other performs specific interactions within an application. Agent S [11] proposes an open-agent framework centered on experience-augmented planning that combines externally retrieved knowledge with local memory. Although existing studies have achieved notable results, they still have limitations. For one, prevailing approaches depend heavily on accessibility trees for screen comprehension instead of visual representations. This data tends to be verbose, noisy, inconsistent, and incomplete, which not only substantially increases computational costs but also feeds considerable distracting information into the MLLM. For another, these methods often fail to model the complex dependencies between subtasks, resulting in restricted performance on real-world in-application complex tasks and cross-application, multi-step tasks.

To overcome the above limitations, we propose Win-Agent, a multi-agent framework featuring two core designs: (1) We innovatively designed a Multi-Dimensional Perception Module

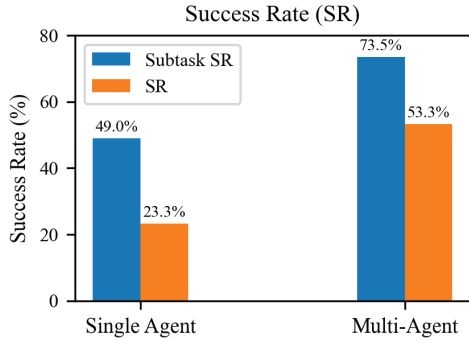


Fig. 1. Performance comparison between single-agent and multi-agent methods on complex tasks.

(MDPM) based on a collaborative small-large model architecture. This module enhances the GUI agent’s fine-grained perception of interface elements from a visual perspective, thereby eliminating its reliance on accessibility trees. (2) we propose a multi-agent architecture with planning, execution, and reflection. In this architecture, the Planning Agent (PA) breaks down lengthy and complex user requests into parameterized subtasks. This process reduces operational steps and simplifies decisions. The PA also manages information flow between subtasks to handle complex dependencies. The Execution Agent (EA) makes step-by-step action decisions based on perceived interface information, records progress, and interacts with the GUI to complete the current subtask. The Reflection Agent (RA) evaluates the correctness of each executed action and provides feedback to the EA for timely error correction. Fig. 2 illustrates the entire process. Through this collaborative multi-agent architecture, Win-Agent can not only robustly handle lengthy operation sequences across applications but also maintain complex dependencies among subtasks. These combined abilities make Win-Agent highly capable of tackling complex user requests in the Windows scenarios.

Our main contributions are summarized as follows: (i) We propose a Multi-Dimensional Perception Module (MDPM), within which we design and train GUI element detection models based on YOLO and OCR. This equips Win-Agent with robust interface element perception capabilities, representing the core breakthrough of this work. (ii) We design a planning-execution-reflection multi-agent framework that resolves the bottlenecks of single agents in handling cross-application complex tasks. This framework can manage the complex dependencies between subtasks that were overlooked in previous studies. (iii) To better evaluate agents on complex tasks, we create the Windows-Eval benchmark. This benchmark contains 150 complex user requests from real-world scenarios. We conduct comprehensive experiments on this benchmark. Results show that our Win-Agent significantly outperforms existing methods in completing complex Windows tasks. We believe our work provides new insights for developing GUI agents in Windows environments.

II. RELATED WORK

The emergence of Multi-modal Large Language Models (MLLMs) has accelerated the development of GUI agents. As noted by Cheng et al. and Xie et al., accurately perceiving GUI elements is a core capability for these agents [7], [8]. Some existing approaches rely on textual metadata such as HTML/DOM or accessibility trees for interface understanding [12], [13]. While easy to parse, these representations often suffer from significant noise, verbosity, and structural incompleteness. Consequently, some research has shifted towards vision-centric paradigms. For instance, UGround trains a visual grounding model using large-scale synthetic data and a modified LLaVA architecture to locate elements from visual input [9]. Although this line of work eliminates the need for accessibility trees, it requires extremely large models and massive datasets to achieve high performance, resulting in substantial computational costs. In contrast, our proposed Multi-Dimensional Perception Module (MDPM) employs a collaborative approach between small and large models. This design avoids reliance on accessibility trees while maintaining a favorable balance between accuracy and efficiency.

Current research on GUI agents primarily focuses on web [14], [15] and mobile platforms [16], [17]. However, desktop computing environments typically involve more complex user requests than these scenarios, demanding stronger planning and decision-making capabilities from agents. OSCAR proposes a unified OS-control agent with state-aware reasoning and replanning, enhancing task robustness through explicit modeling of the OS state graph [18]. For the Windows environment, UFO introduces a hierarchical dual-agent framework where a high-level agent performs task planning and a low-level agent executes actions using screenshots and control information [10]. Agent-S employs an experience-augmented hierarchical planning mechanism, addressing planning and knowledge shortages in long tasks via external knowledge retrieval and internal experience replay [11]. These approaches have significantly advanced task success rates, but they commonly fail to account for intricate dependencies between subtasks, causing severe accuracy degradation on certain cross-application complex tasks. Our proposed Win-Agent specializes in processing complex user requests in Windows OS scenarios. Through its planning-execution-reflection multi-agent framework, it not only maintains complex dependencies between subtasks but also robustly handles lengthy action sequences.

III. WIN-AGENT

A. Task Formulation

Given a user instruction I , the GUI agent (denoted as σ) first obtains an interface observation Z_k before each decision step. Based on the current observation, the action history H_k up to step k , and the user instruction, the agent generates an action α_k for the current step through internal reasoning

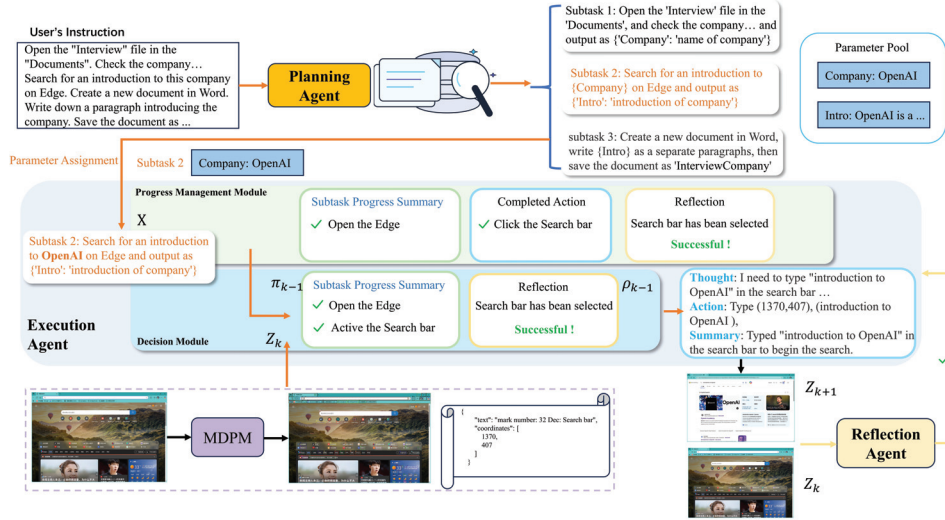


Fig. 2. Schematic illustration of Win-Agent.

and planning, and then interacts with the GUI. This decision-making process can be formalized as:

$$\alpha_k = \sigma(I, Z_k, H_k). \quad (1)$$

In this formulation, Z_k denotes the interface observation at step k , and α_k represents the action to be executed at step k . The architecture of our proposed Win-Agent multi-agent framework is shown in Fig. 2, and will be elaborated in the following sections.

B. Multi-Dimensional Perception Module

To enhance the ability of MLLM-driven GUI agents in accurately locating GUI elements, we designed a Multi-Dimensional Perception Module (MDPM) that combines small and large models. This module utilizes small computer vision models such as YOLOv8 and OCR to perceive GUIs from different dimensions. Specifically, for common text-free GUI controls like pure icons, we train a lightweight YOLO detector to obtain their precise location information and category labels. Fig. 3 presents the model's training accuracy results on the validation set. To ensure robustness across different devices and resolutions, we constructed a cross-device, cross-resolution GUI dataset using actual screenshots from various computer screens, which were annotated and used for YOLO training. This significantly improved the generalization capability of our YOLO detector against screen characteristic differences and resolution variations. For controls with text, OCR is applied to aggregate text blocks and their containers into unified interface elements based on alignment and containment, enabling precise positioning of the controls.

Following that, we employ the Set-of-Marks (SoM) method [19] to add numbered bounding boxes to each identified control on the screenshot. YOLO labels and the text recognized by OCR serve as descriptions for the GUI elements (explaining their functions). This descriptive information, along with the

precise coordinates and the numbered bounding boxes of the elements, is structured into a one-to-one correspondence. This structured perceptual information, together with the original and the marked screenshots, is then fed into the Execution Agent to help it accurately understand the function and location of each GUI element.

To handle tasks requiring precise text actions, for example, "underline the first paragraph in this Word document," our approach also integrates small and large models. First, we leverage the powerful cross-modal fusion capability of the MLLM to identify the start and end lines of the target text based on the user's instruction. Then, OCR determines the specific coordinates of the text within this range, enabling subsequent precise operations like mouse dragging for selection. A detailed case is shown in Fig. 4.

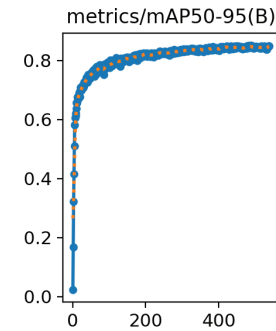


Fig. 3. Model accuracy on the validation set.

C. Planning Agent

We designed a Planning Agent (PA) with the capability of instruction decomposition. This agent is responsible for the overall orchestration and scheduling of tasks. Its primary objective is to convert complex user instructions into a sequence

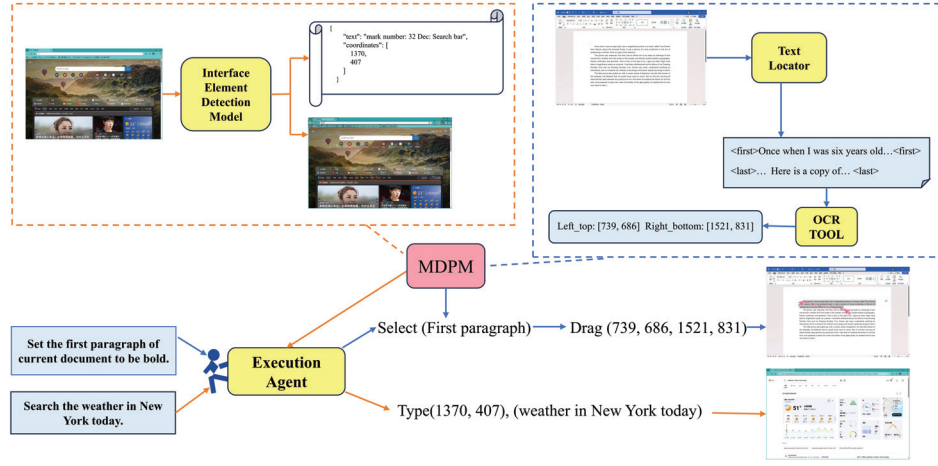


Fig. 4. Illustration of the Multi-Dimensional Perception Module. For GUI controls, a GUI element detection model is used to obtain the location and functional description of each control. For textual content, a text localization module together with an OCR tool is used for perception.

of executable, communicable, and coordinated subtasks. The PA is also responsible for maintaining information transfer and dependency relationships across subtasks during execution.

Instruction Decomposition. For a complex user request, the PA begins by decomposing it into a sequence of parameterized subtasks. These subtasks can be executed independently by the Execution Agent (EA) after their parameters are instantiated. This significantly shortens the operation sequence for each task and lowers the complexity of decisions. Moreover, it also avoids the noise introduced by performing lengthy reasoning over the entire global history. Furthermore, to substantially reduce hallucinations and dependency omissions in long-chain planning, we introduce experience-augmented planning via retrieval. A knowledge base for task decomposition is built using authentic interaction logs. In the decomposition stage, the most similar prototype is retrieved and undergoes structured adaptation. This shifts the planning paradigm from zero-shot to one that is informed by prior knowledge.

Inter-Subtask Communication and Dependency Management. In real-world scenarios, subtasks often have close dependencies. The PA centrally manages these dependencies and maintains a parameter pool to support the generation, caching, and downstream reuse of parameters. When a subtask is completed successfully, its output parameters are written into the parameter pool. Subsequent subtasks then read the required parameters directly from this pool during their parameter assignment. This reliably fulfills the dependencies between subtasks. Based on the direction of dependencies, common subtask types can be categorized into four classes: (a) Producer Subtask: its execution results are assigned to parameters in subsequent subtasks for their use. (b) Consumer Subtask: its parameter values come from the output parameters of previous subtasks. (c) Producer-Consumer Subtask: It both depends on the results of prior subtasks and produces intermediate results for subsequent ones. (d) Independent Subtask: It has no dependencies on other subtasks. Via this decomposition and communication mechanism, the PA converts complex tasks

into sequences of subtasks with logical dependencies. Using the parameter pool as a medium, it maintains data consistency and execution controllability across subtasks. Consequently, a clear and stable context is laid for the ensuing execution and reflection phases.

D. Execution Agent

Once the Planning Agent (PA) completes task decomposition and subtask instantiation, it transfers the executable subtasks to the Execution Agent (EA). The EA consists of a Progress Management Module and a Decision Module. These two modules work in coordination to execute the subtasks.

The Decision Module is the core component responsible for generating actions and interacting directly with the GUI. It performs step-by-step reasoning and action generation for the current subtask X . At step k , the module first obtains the perceptual information Z_k of the current interface from the MDPM. It then fuses this information with the current subtask progress π_{k-1} , aggregated by the Progress Management Module, and the reflection ρ_{k-1} provided by the Reflection Agent (RA). Based on this fused information, the Decision Module reasons to determine the action α_k to be executed. It adopts a chain-of-thought strategy: first generating a reasoning chain for the current step, then outputting a structured action decision to facilitate parsing. This method not only helps the Decision Module make better decisions but also assists the RA in evaluating execution outcomes. The above process can be formally represented as:

$$\alpha_k = \Phi(X, Z_k, \pi_{k-1}, \rho_{k-1}). \quad (2)$$

We integrated a dedicated action execution toolbox for the EA. This toolbox, developed using pywinauto, covers most control operations in GUIs. The generated structured action decisions are then parsed into specific action types and corresponding parameters. These are matched with the specific tools in the action execution toolbox, and the parameters are passed to them to perform the corresponding mouse

and keyboard operations. Essentially, the toolbox materializes action decisions into physical execution on the application.

The Progress Management Module is responsible for incrementally tracking and summarizing the execution progress of the current subtask. We design it at the subtask level rather than the entire instruction level for two reasons: (1) It provides the Decision Module with a clear operation history and current subtask progress. This allows the Decision Module to better determine the next action and complete the current subtask more effectively, avoiding interference from lengthy and complex instruction-level progress information. (2) By tracking the progress of each subtask individually, the module enhances the accuracy of progress perception and avoids the illusions that can arise from long-term tracking of the entire instruction's execution. Its input at step k includes the subtask X delivered by the Planning Agent, the previous progress summary π_{k-1} , the completed action α_k at this step, and the reflection ρ_k provided by the Reflection Agent after the action is executed. The module outputs the updated progress π_k based on this information:

$$\pi_k = \Psi(X, \pi_{k-1}, \alpha_k, \rho_k). \quad (3)$$

E. Reflection Agent

In long-horizon, multi-step tasks, even advanced MLLMs can make mistakes in perception and reasoning. A mistake at any step could cause the overall task to fail. To mitigate this issue, we introduce a Reflection Agent (RA) driven by an MLLM. The RA immediately verifies the outcome after each action is executed and generates structured reflection feedback. This ensures that errors can be detected promptly and corrected in subsequent steps.

Following every action performed by the EA, the RA assesses the outcome. It does so by comparing the screen states before and after the action, while taking into account the current subtask and its progress status, to judge if the outcome matches the goal and to detect any invalid responses or errors. The reflection generation process of the RA is formalized as:

$$\rho_k = \Gamma(X, \alpha_k, Z_{k-1}, Z_k). \quad (4)$$

Based on the above judgment, the reflection from the RA is sent to the EA. The EA then decides whether to correct the error or proceed to the next action.

IV. EXPERIMENTS

A. Experiment Setting

Benchmark Dataset. Although existing benchmarks designed for Windows environments, such as WindowsAgentArena [20] and WindowsBench [10], are substantial in size, their tasks primarily consist of basic operations and rarely involve cross-application instructions. This presents a discrepancy with the multi-step dependencies and fine-grained operations of actual practice. Therefore, to systematically evaluate the capability of our Win-Agent in executing complex intra-application and cross-application tasks within an authentic Windows environment, we

constructed a benchmark dataset named Windows-Eval. This dataset comprises 150 complex user requests, covering popular Windows applications from daily use spanning scenarios like productivity, social interaction, and entertainment. Each user request consists of interdependent subtasks, emphasizing cross-page coordination, fine-grained editing, and long-horizon decision-making. To ensure task diversity and challenge, two annotators with relevant expertise collaboratively designed and cross-checked all instructions.

Evaluation Metrics. Given the significant variations in page layouts and success criteria across different subtasks, developing automated evaluation scripts for each subtask would be prohibitively expensive. Therefore, this study employs manual evaluation and quantifies performance using two metrics: Success Rate (SR) and Subtask Success Rate (SSR). The Success Rate (SR) is defined as the ratio of tasks that the agent successfully completes to the total number of tasks. The Subtask Success Rate (SSR) is defined as the ratio of subtasks completed by the agent to the total number of subtasks.

Baseline Models. In our experiments, we include three categories of baseline models: high-performing MLLMs; single-agent systems capable of automating Windows GUI tasks; and multi-agent systems. For the MLLM category, we select leading models such as Qwen2.5-VL. Since these models cannot directly interact with the GUI, we employ prompt engineering to provide them with a comprehensive set of executable actions. The models are guided to output their intended operations, which are then executed by a human operator, thereby enabling them to function as agents. For the single-agent baseline, we choose the open-source agent SeeClick. For the multi-agent baseline, we select UFO, a GUI agent specifically designed for Windows, and Agent-S, a multi-agent framework for Windows GUI task automation. In all experiments, we use GPT-4o as the computational engine for every agent. We selected EasyOCR as the OCR tool for Win-Agent.

B. Results

As illustrated in Table I, a performance comparison has been conducted between Win-Agent and other agents on the Windows-Eval platform. We observe that GUI agents driven solely by a single MLLM without collaborative modules perform poorly in almost all tasks. Even the top-performing agent, Qwen2.5-VL, and the second-best, GPT-4o, did not achieve a success rate above 20%, reaching only 15.3% and 10.7%, respectively. The underperformance of these GUI agents can be attributed to one key factor. Due to their inherent technical limitations, MLLMs struggle to accurately perceive the positions of GUI elements.

Our analysis of SeeClick, a single agent with enhanced GUI localization capabilities, shows that while it outperforms Qwen2.5-VL on Windows-Eval, its task success rate remains low. This result demonstrates that unoptimized single agents may struggle to effectively plan, reason, and accomplish these complex tasks.

UFO was the previous state-of-the-art GUI dual-agent framework specifically designed for Windows, and Agent-S is a representative multi-agent framework in the PC context. On Windows-Eval, although they show improved performance over single agent approaches, our Win-Agent remains the top-performing framework. Through further analysis, we can discern their limitations in both screen comprehension and planning reasoning: (1) Existing methods perceive the screen by obtaining the accessibility tree. However, this type of data is often lengthy and suffers from noise, inconsistency, and incompleteness. This not only increases computational cost but also introduces incorrect and distracting information to the MLLM. (2) These methods are inadequate at managing the dependencies between subtasks within a complex instruction, especially when the execution of a later subtask is conditioned on the result of an earlier one.

In contrast, our proposed MDPM enhances Win-Agent’s fine-grained interface perception at the visual level. This enables Win-Agent to parse complex interfaces and perform precise operations on interactive elements and text. It also addresses the performance limitations in existing methods caused by their reliance on accessibility trees. Furthermore, through our planning-execution-reflection multi-agent architecture, Win-Agent gains the capabilities for complex task decomposition, inter-subtask information passing, progress tracking, and timely error correction. These competencies enable Win-Agent to handle complex user requests with high efficiency. Consequently, our Win-Agent substantially outperforms other state-of-the-art methods, achieving a Success Rate improvement of 6% over Agent-S. Of course, the fact that Win-Agent currently cannot complete some instructions in Windows-Eval also confirms the necessity of continued exploration in complex Windows scenarios and underscores the challenging nature of this benchmark.

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT AGENTS ON THE
WINDOWS-EVAL BENCHMARK

Agent	Type	SSR	SR
GPT-4o	MLLM	40.6%	10.7%
Claude-3.5	MLLM	16.1%	2.0%
Gemini-2.0	MLLM	33.3%	8.0%
Qwen2.5-VL	MLLM	45.5%	15.3%
SeeClick	Single-Agent	49.0%	23.3%
UFO	Multi-Agent	61.2%	38.0%
Agent-S	Multi-Agent	70.0%	47.3%
Ours	Multi-Agent	73.5%	53.3%

C. Ablation Study

To quantify the contribution of each core component in Win-Agent, we conducted ablation studies on the MDPM, the

TABLE II
ABLATION STUDY OF WIN-AGENT COMPONENTS ON THE
WINDOWS-EVAL BENCHMARK DATASET

Ablation study			SSR	SR
MDPM	Planning Agent	Reflection Agent		
✓	✓	✓	47.1%	16.7%
✓	✓	✓	59.0%	24.7%
✓	✓	✓	51.2%	23.3%
✓	✓	✓	73.5%	53.3%

PA, and the RA. The results are presented in Table II. (1) A comparison of rows 1 and 4 reveals the most critical role of the MDPM. Without the MDPM, the SSR drops from 73.5% to 47.1%, and the SR from 53.3% to 16.7%. This suggests that agents relying only on the native abilities of the MLLM fail to precisely perceive GUI elements and grasp the meaning of interactive controls. In contrast, the structured perception from the MDPM greatly improves the agent’s accuracy in understanding the interface. (2) The contrast between rows 2 and 4 demonstrates that the PA is essential for handling complex instructions. After removing the PA, the SSR decreases from 73.5% to 59.0%, and the SR from 53.3% to 24.7%. Without the PA, complex instructions are treated as a single long-horizon task, making it hard to effectively manage dependencies between subtasks. This significantly weakens progress perception and overall decision-making capability, making it difficult to convert subtask success into instruction-level success. (3) Rows 3 and 4 reveal that the RA is crucial for the robustness of Win-Agent in long-horizon tasks. When the RA is removed, the SSR falls from 73.5% to 51.2%, and the SR from 53.3% to 23.3%. Without a reflection mechanism, the system struggles to promptly identify and correct intermediate errors, often leading to redundant or incorrect operations, which significantly reduces the task completion rate.

D. Case Study

We presented Win-Agent with a complex request, as shown in Fig. 5. First, Win-Agent broke down this request into five subtasks with dependencies. It then proceeded to execute these subtasks sequentially. The results from executing subtask 1 and subtask 2 were stored in a parameter pool. These results were subsequently used by subtask 4 and subtask 5, respectively. The successful execution of this task demonstrates Win-Agent’s strong capabilities in complex task planning and interface perception.

V. CONCLUSION

This study presents Win-Agent, a multi-agent framework for GUI automation that features accurate interface perception and can process complex user instructions in Windows scenarios. Win-Agent incorporates a Multi-Dimensional Perception Module (MDPM). The MDPM leverages a collaboration between small and large models. This allows Win-Agent to achieve fine-grained perception of interface elements and text using only visual observations. Consequently, it eliminates the need for lengthy and noisy accessibility trees. By coordinating

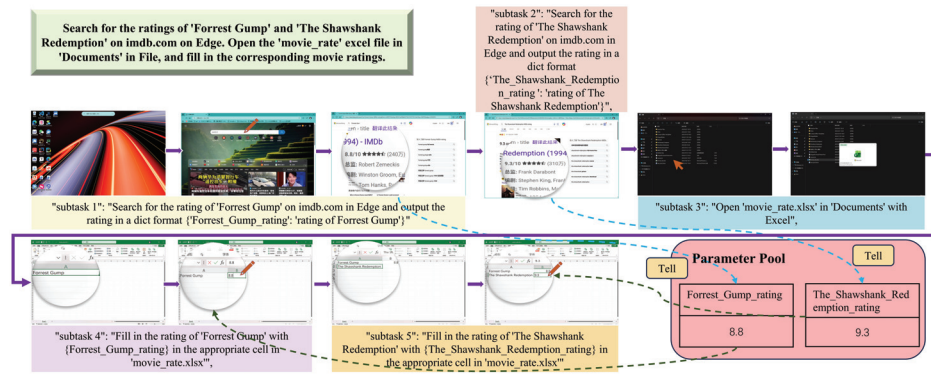


Fig. 5. Example of Win-Agent executing a complex user request across multiple applications.

the Planning, Execution, and Reflection Agents, it reduces the decision-making complexity of complex Windows tasks, overcomes the planning limitations of single agents in long-horizon, cross-application tasks, and effectively manages dependencies among subtasks. We construct a new benchmark, Windows-Eval, comprising 150 complex user requests from real-world settings. Experimental results confirm that Win-Agent outperforms existing state-of-the-art models on complex Windows tasks. Our research is expected to provide new insights for developing GUI agents in Windows environments.

Nevertheless, Win-Agent is still unable to execute certain instructions within Windows-Eval. This shows that developing a universal GUI agent for complex Windows environments continues to be an open challenge. Additionally, this study concentrates mainly on automating tasks for widely-used Windows applications. For future research, we will enhance the generalization capability of the MDPM to unseen applications and interface styles, and also extend and refine the evaluation benchmark. The code and detailed information about Windows-Eval will be made publicly available at: <https://github.com/liuyao2001/Win-Agent>.

ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China (No. 12471500)

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024.
- [2] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, "A survey on multimodal large language models," *National Science Review*, 2024.
- [3] A. Adekotoju, A. Odumabo, A. Adedokun, and O. Aiyeniko, "A comparative study of operating systems: Case of windows, unix, linux, mac, android and ios," *International Journal of Computer Applications*, 2020.
- [4] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "Appagent: Multimodal agents as smartphone users," in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025.
- [5] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024.
- [6] Q. Wu, K. Cheng, R. Yang, C. Zhang, J. Yang, H. Jiang, J. Mu, B. Peng, B. Qiao, R. Tan *et al.*, "Gui-actor: Coordinate-free visual grounding for gui agents," *arXiv preprint arXiv:2506.03143*, 2025.
- [7] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei *et al.*, "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments," *Advances in Neural Information Processing Systems*, 2024.
- [8] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, and Z. Wu, "Seelick: Harnessing gui grounding for advanced visual gui agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- [9] B. Gou, D. R. Wang, B. Zheng, Y. Xie, C. Chang, Y. Shu, H. Sun, and Y. Su, "Navigating the digital world as humans do: Universal visual grounding for gui agents," in *International Conference on Representation Learning*, 2025.
- [10] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan *et al.*, "Ufo: A ui-focused agent for windows os interaction," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025.
- [11] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. Wang, "Agent s: An open agentic framework that uses computers like a human," in *International Conference on Representation Learning*, 2025.
- [12] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *International Conference on Representation Learning*, 2024.
- [13] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- [14] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," *Advances in Neural Information Processing Systems*, 2023.
- [15] Q. Chen, D. Pitawela, C. Zhao, G. Zhou, H.-T. Chen, and Q. Wu, "Webvln: Vision-and-language navigation on websites," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [16] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala, D. Toyama, R. Berry, D. Tyamagundlu, T. Lillicrap, and O. Riva, "Androidworld: A dynamic benchmarking environment for autonomous agents," in *International Conference on Representation Learning*, 2025.
- [17] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding *et al.*, "Cogagent: A visual language model for gui agents," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [18] X. Wang and B. Liu, "Oscar: Operating system control via state-aware reasoning and re-planning," in *International Conference on Representation Learning*, 2025.
- [19] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v," *arXiv preprint arXiv:2310.11441*, 2023.
- [20] R. Bonatti, D. Zhao, F. Bonacci, D. Dupont, S. Abdali, Y. Li, Y. Lu, J. Wagle, K. Koishida, A. Buckner, L. K. Jang, and Z. Hui, "Windows agent arena: Evaluating multi-modal OS agents at scale," in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.